



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure
E - Environment
A - Actuators
S - Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.walls  
self.food_positions  
self.opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline environment is Multi-Agent because the environment contains `self.opponents`, which represent additional entities that move independently and can interact with the main agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the sequence of observation or percepts that an agent has received from the environment over time. It represents the information available to the agent from its interaction with the environment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

S - Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Partially Observable because the agent does not receive the complete environment state through `get_percept()`. It can see its own positions, but it does not receive the complete locations of all food or all walls.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

P - Performance Measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

A performance measure should evaluate the agent's actual outcomes in the environment rather than its internal processing effort. Rationality is concerned with selecting actions that produce the best possible or expected outcomes, so external results provide a meaningful basis for evaluating the agent.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding toxic traps from the sensors increases the amount of hidden information in the environment. The environment remains Partially Observable, because the agent cannot observe the complete environment state.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.
10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

`smells_toxin` expands the agent's percept sequence by providing information about toxic locations. This allows the agent to identify a dangerous state and choose actions that reduce the expected penalty, helping it act more rationally by maximizing expected utility.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.
11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit demonstrates that an agent can exploit a badly designed performance measure. If the system rewards performing an action rather than achieving the intended goal, the agent may repeatedly perform useless actions simply to increase its score. Therefore, the performance measure must reward desirable outcomes rather than meaningless activity.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING